

Week 1

OPEN SOURCE CONCEPTS

Course Facebook group



Open Source-2024- Mansoura FCI Course

Open Source-2024- Mansoura FCI Course

<https://www.facebook.com/groups/349595874642631>



Introduction

- The term “open source” refers to software in which the source code is freely available for others to view, amend and adapt.
- **Freeware:** There do exist software which are actually "free" in the sense of price. These are known as **Freeware**. Lots of freeware can be downloaded from the internet for various different purposes such as currency converters, drawing graphs and charts etc. But freeware may not come with the source code. **Therefore freeware differ from free software.** The focus in this chapter is on free software rather than freeware.
- **FREE SOFTWARE** means the software is freely accessible and can be freely used, changed, improved, copied and distributed by all who wish to do so. And no payments are needed to be made for FREE SOFTWARE.
- *Note: Freeware and Free S/W are two different things. Freeware are the S/w which permit redistribution but not modification and their Source Code is not available . These packages are not free software; so don't use the word 'freeware' to refer to free Software.*
- **Shareware** is software, which is made available with the right to redistribute copies, but it is stipulated (required) that if one intends to use the software often after a certain period of time, then a license fee should be paid.

Free Software (4 freedoms)

four freedoms to users:

- Freedom 1: The freedom to **run** the program for any **purpose**.
- Freedom 2: The freedom to **study how the program works**, and change it to make it do what you wish.
- Freedom 3: The freedom to redistribute **copies** so as to help your neighbour.
- Freedom 4: The freedom to **improve** the program, and release your improvements (and modified versions in general) to the public, so that the whole community benefits.

Free Software (Terminologies)

- **FLOSS**: A software which is free as well as open belongs to category FREE /LIBRE/ OPEN SOURCE SOFTWARE.
- **FSF**: is Free Software Foundation. It promotes the development and use of free software. It is an non- profit organization.
- **OSI**:Open Source Initiative, is a non-profit corporation dedicated to managing and promoting the Open Source definition by certifying products.
- **W3C**: is acronym for World Wide Web Consortium. It is responsible for producing the software standards for world wide web.
- **Proprietary S/W**: is the software that is neither open nor freely available. Its use is regulated and modification is either forbidden or requires special permission by the supplier.

Examples of FLOSS/FOSS :

Operating systems and Desktop environments:

- **Linux** - Operating system kernel
- **Ubuntu** - Linux distribution with full complement of software for everyday use.
- **Android smart** - phone operating system - by Google / Open Handset Alliance

Graphics and multimedia:

- **GIMP** - Bitmap graphics editor, similar to Adobe Photoshop
- **Blender** - Advanced 3D modelling and rendering application.
- **Songbird** - similar to iTunes with built-in browser.
- **Audacity** - digital audio editor

Examples of FLOSS/FOSS ...

Office software :

- **OpenOffice.org** - office productivity software. Comparable to Microsoft Office. It uses an open file format and can read and write Microsoft Office files
- **NeoOffice** - Mac OS X version of OpenOffice
- **PDFCreator** - creates PDFs from any Windows program.

Internet related software

- **Apache webserver** - web server
- **Mozilla Firefox** - web browser
- **Mozilla Thunderbird** - mail client
- **Google Chrome** - Google's web browser

Examples of FLOSS/FOSS ...

Programming related :

- **Eclipse** - software framework and Java IDE
- **PHP** - server-side programming language
- **PERL** - Dynamic programming language
- **Python** - versatile, clean and powerful programming language used for cross platform desktop applications, server-side scripting for websites, and scripting within java and .net environments
- **MySQL** - Database management system
- **Java** - Programming language
- **NetBeans** :refers to both a platform framework for Java desktop applications, and an IDE for developing applications with Java, JavaScript, PHP, Python, Ruby, C, C++, and others.

Others:

- **Celestia** - 3D space simulation software.
- **Flight Gear** - flight simulator
- **Second life** - virtual world viewer

Type of standards

1. **Proprietary Standards**: are those for which users have to buy license to use them. These are also known as closed or exclusive standards. Standards like Microsoft Office Formats (eg. .doc,. docx,. ppt,. xls) or Media formats (e.g. .wma, .wmv etc.), Apple Media Formats (Such as .mov) are proprietary standards as they are the property of their respective owners.
2. **Open Standards**: are internationally accepted technical standards that guarantee that data can be exchanged or is accessible across platforms and application. Simply we can say that the specification of open standards is open to all without any restriction.
 1. **Common open standard formats**:
JPEG (Joint Photographic Expert Group),
PNG(Portable Network Graphic),
SVG(Scalable Vector Graphic),
FLAC(Free Lossless Audio Codec)
ODF(Open Document Format),
OGG(Ogg Vorbis),

Open Source software Licensing

What is a license?

- .Grants permission to use a copyrighted work
- .Can grant any or all of the rights associated with copyright
- .Can impose other restrictions, such as type or place or usage, or duration of the license
- .Does not transfer ownership of the copyright
- .A **software license** is the contract between the software owner and the licensee defining terms of use of software

Open Source Software Licenses

- Source Code

- .Source code to original product always provided;
- .Licensee **can modify or enhance** source code (create “**derivative works**” مشتق)
- .or include source code with other license types (create “**larger works**”);
- .Licensee may be required to **share** modifications with the world (in source and/or binary form), but not necessarily.

Common Properties of Open-Source Software

1. Free **redistribution** of the software: the software can be sold or given away as a product or included in a software package. This can be done without paying any royalties.
2. The **source code** of the licensed software is either included in the distribution, or at least there are well-publicized means of obtaining the source code. This source code is usable for developing modified versions of the software.
3. **Creation of derived works are allowed**, and the license allows them to be distributed under the same terms and license as the original software. Depending on the original software's license, in some cases these derived works must be differentiated from original software by changing their name or version number, or can be only distributed in the form of source-code patches.
4. The **software can be used by any person** or group of people, and in any fields of endeavor, with no limitations.

CopyLeft

- **The opposite of copyright.** Where copyright is used to protect intellectual property (including software) from being copied or distributed,
- **copyleft** is used to ensure that open-sourced intellectual property and software **can be copied** or distributed as *open* source.
- According to its strength, there are two kinds of copyleft:
 1. **Strong copyleft:** when works derived from other strong-copyleft licensed works, or linked to these works, ***must continue having strong copyleft licenses***, or even exactly the same license. That is, those open-source works cannot be closed in the future.
 2. **Weak copyleft:** when **works using** weak copyleft licensed works, or **linked** to it, can be licensed **under other licenses**, even **closed-source** licenses. In this case, the ***copyleft only affects the original weak*** copyleft licensed work.
- There are also open-source licenses without copyleft: they simply don't care about future openness of derived software.

Permissiveness

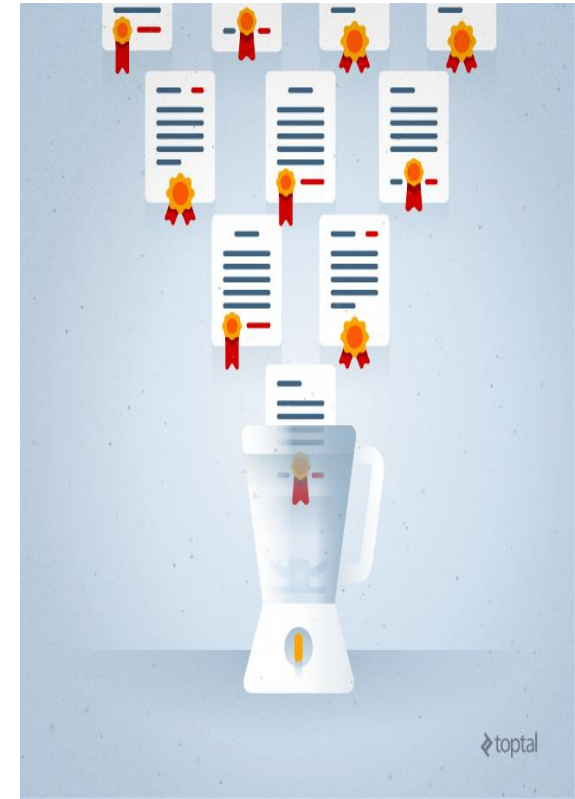
- According to their permissiveness, the licenses can also be classified in:
 1. **Strict licenses:** when **you cannot mix** strong licensed software with **closed-source**, or even with more permissively licensed software.
 2. **Permissive licenses:** when products usually can be **mixed** with **closed-source software**, or software with a totally open-source license.
- Usually, strong copyleft licenses are also strict, and weak copyleft licenses are more permissive, but it doesn't have to be that way.

Open-Source Licenses Common Differences

1. Type of **copyleft**: weak or strong or inexistent.
2. Type of **permissiveness**: permissive or strict.
3. The **obligation** الاضطرار **of adding a copyright notice** in the source code, or in the user interface.
4. Availability of **source code** for users (not just developers), (so that people using a service in the cloud that uses this kind of open-source software should have the choice of downloading the source code of the software)

Problem of Mixing Code with Different Licenses

- some licenses are permissive, allowing the users to combine the code with differently-licensed source code (perhaps with additional conditions). This case would allow to mix this kind of open-source licensed software with closed-source software. An example of this kind of license is MIT License.
- Others are more restrictive, so the source code can only be combined with code that is licensed in a similar way, and the final result must be licensed with the same original license. An example of this kind of license is General Public License (GPL).



Common Open-Source Licenses

<https://www.toptal.com/open-source/developers-guide-to-open-source-licenses>

<https://itsfoss.com/open-source-licenses-explained/>

1-GNU General Public License (GPL)

The core concept behind the GPL being *any* Derivative Work must be released under the GPL too.

Its differential characteristics:

1. **Strong copyleft.**
2. **Very strict** license.
3. It's usually called a '**viral**' منتشرة license: if you link your code to **another piece** of code licensed under the GPL and want to distribute the results, the **whole** product must be GPL-licensed (GPL**ed**).
4. It's also an '**embracing**' اقابلة للخلط مع أنواع أخرى license: if you are developing a software and want to license it under the GPL, you can link it or include other open-source software as long as this software has a license compatible with GPL. It doesn't require any obligation not required by the GPL.
5. The Work is suitable for **commercial** use.
6. **Licensees** can modify the work.
7. Licensees must release the **source** alongside with Derivative Work.

Popular projects- GPL

- The GPL is the natural license for the projects of the Free Software Foundation. Including the [GNU tools](#) at the heart of any Linux system. Large projects — *a fortiori* commercial ones — tend to use the GPL in conjunction with one or several other Licenses.
- [Inkscape](#) (Vector drawing): GPLv2
- [Drupal](#) (Web Content Management System): GPLv2
- [MariaDB](#) (Databases): GPL v2
- [MySQL](#) (Databases): GPL and Commercial License
- [Qt](#) (cross-platform application framework): LGPL, GPL, and Commercial — depending the modules and service-agreement level

2-MIT License

- is a very **permissive**.
- allows everyone to basically use the so-licensed code for whatever you want as long as you **keep the copyright message**, and know that the software comes without warranty ضمان of any kind.
- **Non-copyleft**
- The Work is suitable for commercial use.
- Licensees can **modify** the work.
- Licensees must provide proper **attribution** for the Work.
- Licensees may redistribute **Derivative** Work under different **terms**
- Licensees **do not have** to distribute the **source** code alongside with their Derivative Work.

Popular MIT Application

- [node.js](#) (JavaScript runtime environment): MIT License
- [jQuery](#) (client-side JavaScript library): MIT License (until 2012, dual-license MIT/GPL)
- [Atom](#) (text editor): MIT License
- [AngularJS](#) (JavaScript application framework): MIT License
- [SQLAlchemy](#) (SQL toolkit and Object Relational Mapper for Python): MIT License

3-Mozilla Public License (MPL)

- In the MPL the “**licensing unit**” is the source **file**
- Licensors are not allowed to restrict the user rights and access on any file covered by the MPL. But the same project can also contain proprietary non-MPL licensed files. The resulting project can be released under any license, provided that access to the MPL licensed files is granted.
 1. **Weak copyleft (bound to individual files)**
 2. The Work is suitable for commercial use.
 3. Licensees can modify the work.
 4. Licensees must provide proper attribution for the Work.
 5. Licensees may redistribute Derivative Work under different terms
 6. Licensees cannot relicense MPL-licensed source
 7. Licensees must distribute MPL-licensed source code alongside with their Derivative Work.
- The MPL, in its current version 2.0 is compatible with the GPL. This isn't true for previous versions of the MPL.

MPL popular projects

- Mozilla Firefox(web browser), Mozilla Thunderbird(email client): MPL
- [LibreOffice](#) (office suite): MPL2.0
- [H2 Database Engine](#) (database): MPL2.0 and Eclipse License 1.0
- [Cairo](#) (2D graphic engine): MPL 1.1 or LGPLv2.1

Licenses Comparison

للاطلاع فقط

snyk	Copyleft					Permissive			
									Unlicense
Permissions in addition to commercial use, distribution, modification:									
Patent use	●	●	●	●	●	●	●	●	●
Patent use	●	●	●	●	●	●	●	●	●
Conditions									
Disclose source	●	●	●	●	●	●	●	●	●
License & copyright notice	●	●	●	●	●	●	●	Source	●
Network use is distribution	●	●	●	●	●	●	●	●	●
Same license	●	●	Library	●	File	●	●	●	●
State changes	●	●	●	Some	●	●	●	●	●
Limitations/Disclaimers									
Liability	●	●	●	●	●	●	●	●	●
Warranty	●	●	●	●	●	●	●	●	●
Trademark use	No explicit limitation				●	●	●	●	●

<https://snyk.io/learn/open-source-licenses/>

How do licensors make money with open source software?

Usually by providing other services, such as:

1. Support
2. Training
3. Customization
4. Integration
5. Certification
6. Additional optional services (like data analysis)

An introduction to version control systems with Git



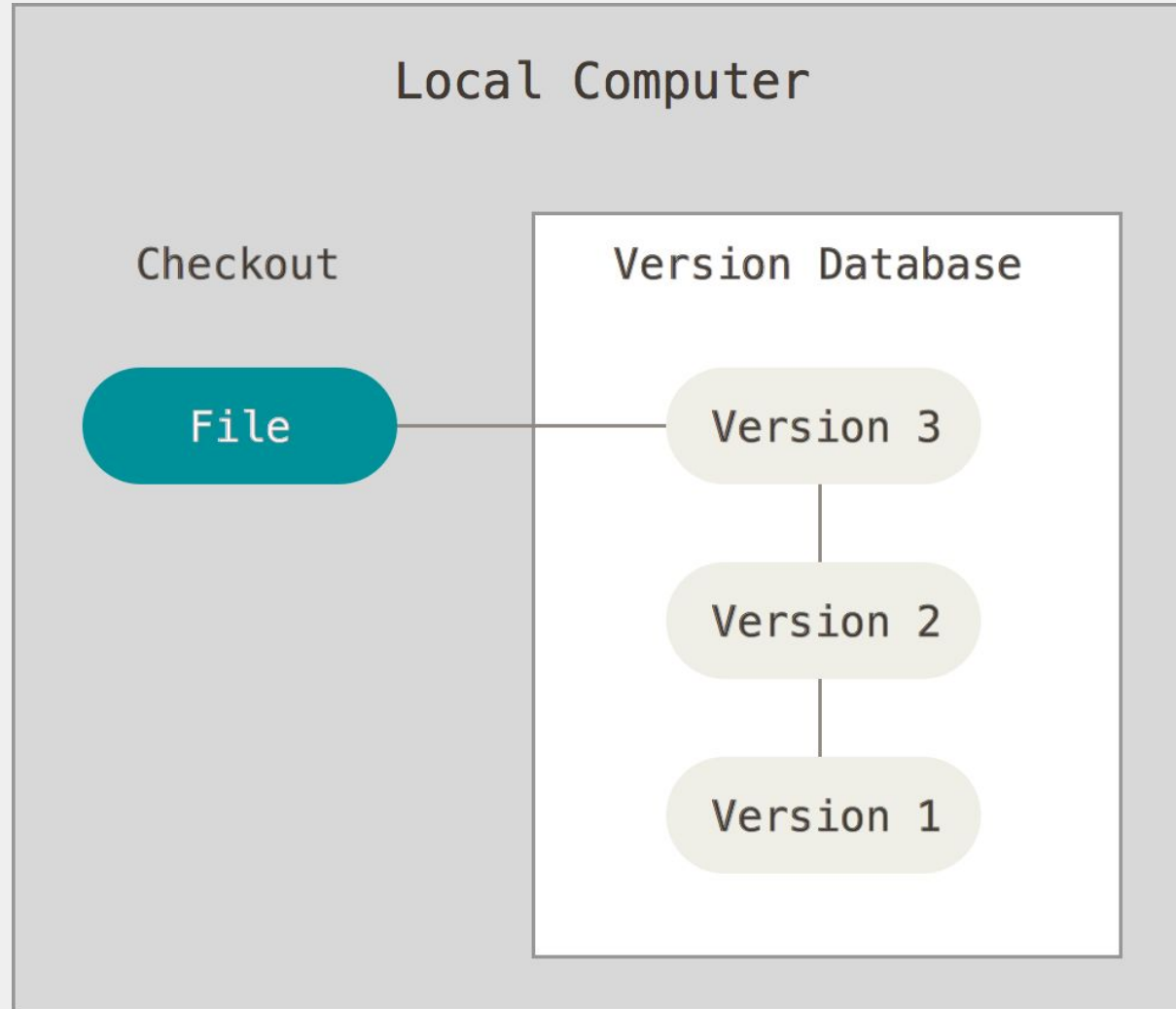
Version control systems

- Version control systems record changes to a file or set of files over time so that you can recall specific versions later
- Many systems have risen to popularity over the years
 - RCS
 - CVS
 - Subversion
- We will focus on Git

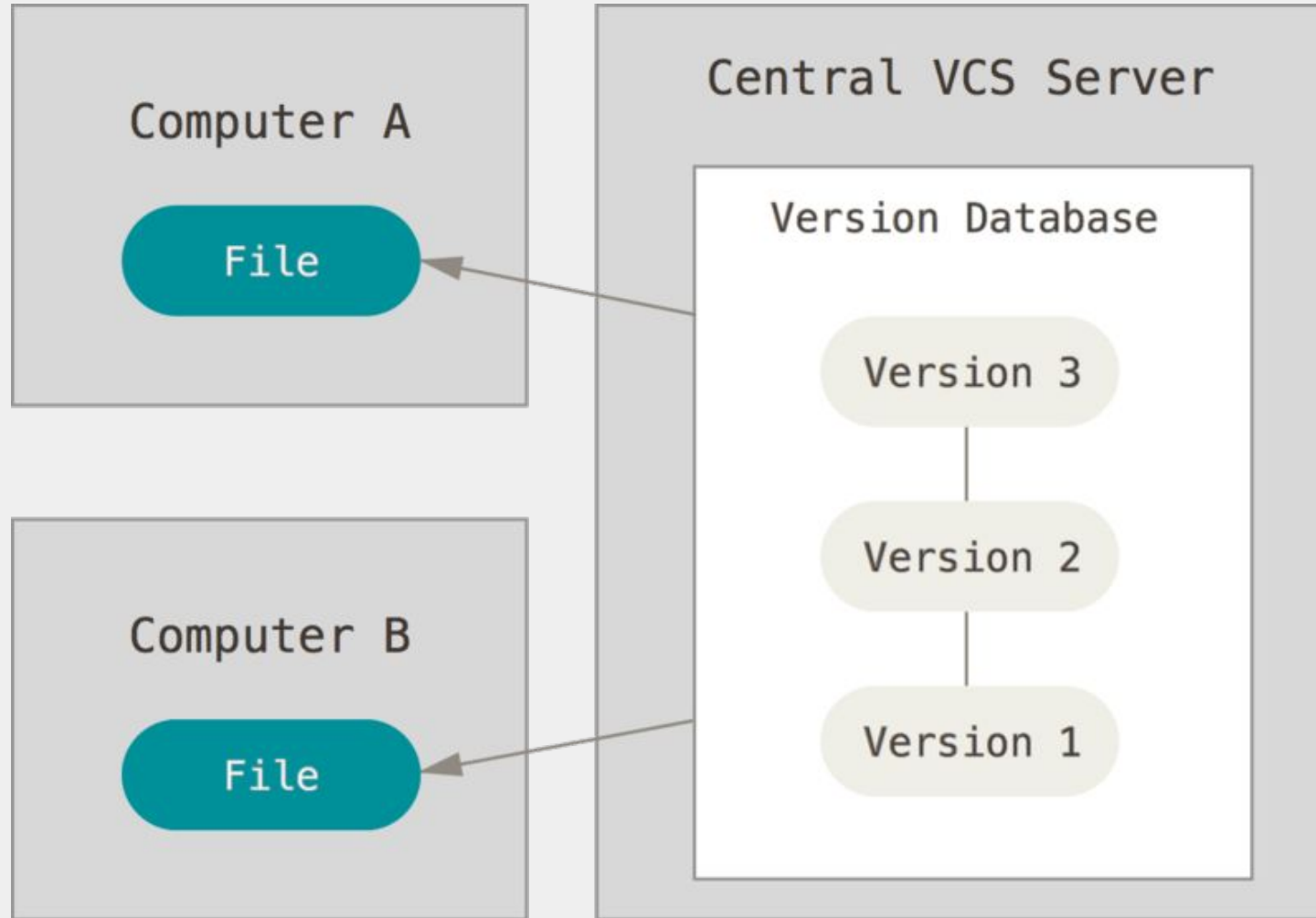
Why use version control?

- These systems help with:
 - Tracking changes
 - Short and long term undo
 - Backup and restore
 - Synchronization
 - Collaboration

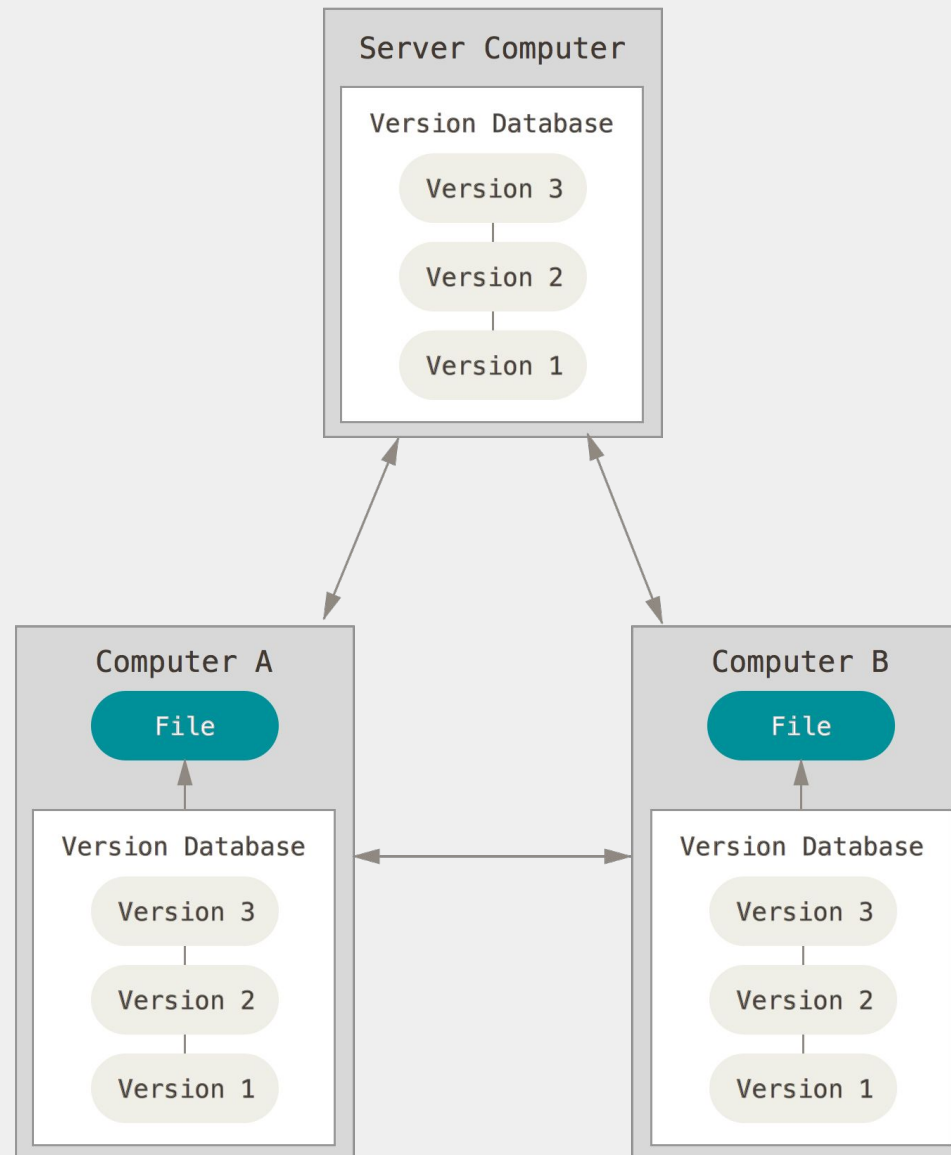
1- Local version control systems



2-Centralized version control systems

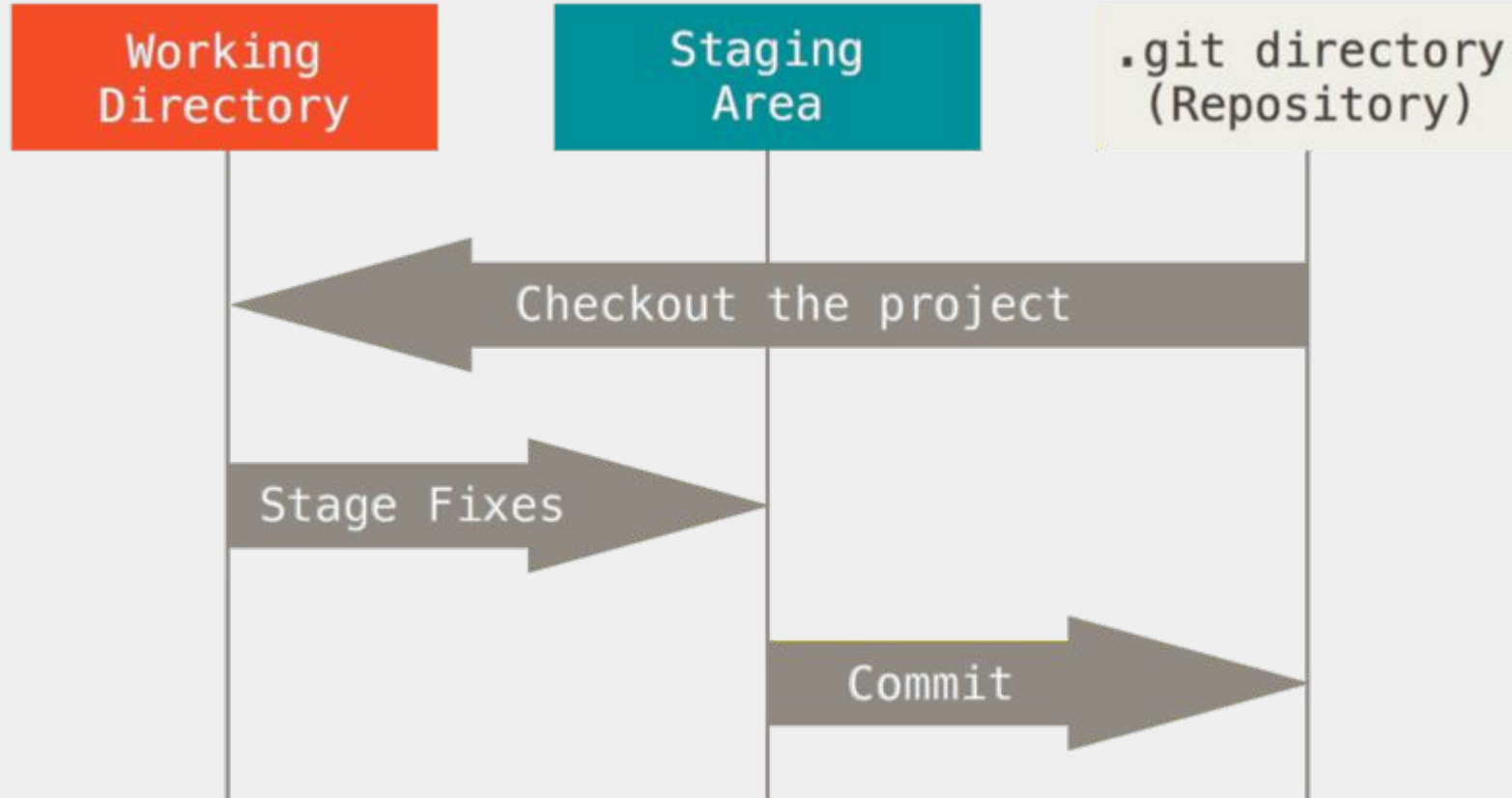


3-Distributed version control systems



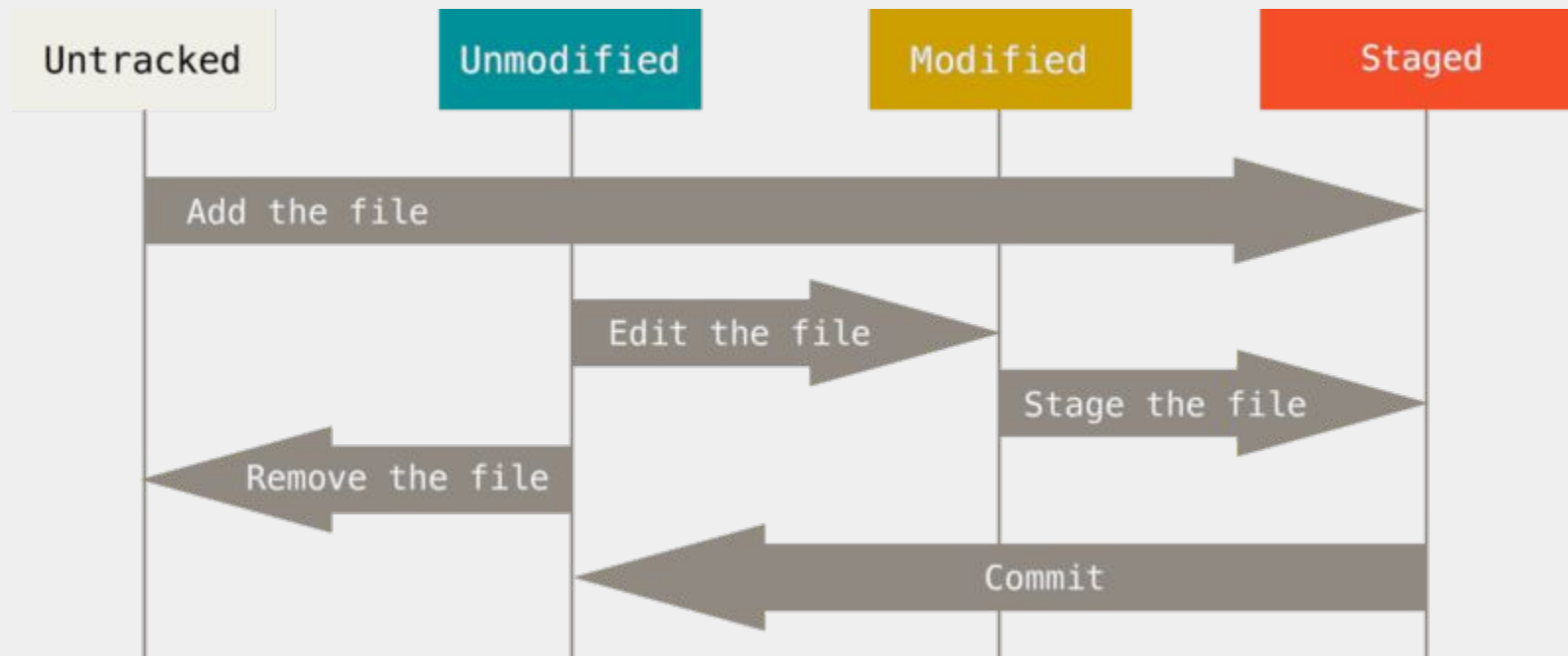
The basic Git workflow

- **Modify** files in your *working directory*
- **Stage** the files, adding snapshots to your *staging area*
- **Commit** your changes to your local copy of the *repository*



The lifecycle of a file in Git

- Git does not necessarily keep track of all files in your working directory



Branches

By default, Git commits go into the **master** branch.

What is a branch?

A branch is nothing but a pointer to the latest commit in the Git repository. So currently our master branch is a pointer to the second commit “demo.txt file is modified”.

Why are multiple branches needed?

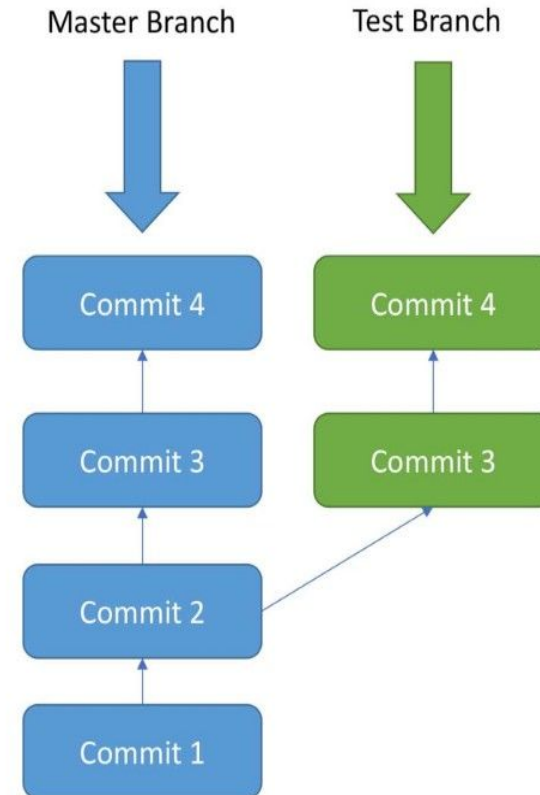
Multiple branches are needed to support multiple parallel developments. Refer the image below to see how branches work.

Branches (Example)

Initially, commit 1 and commit 2 were done in the master branch. After commit 2 a new Branch called as “Test” is created, and commit 3 and commit 4 are added to the test branch.

At the same time, a different commit 3 and commit 4 are added to the master branch. Here we can see that after Commit 2, two parallel developments are being done in 2 separate branches.

The Test Branch and the Master Branch have diverged here and have different code — the code from Test Branch can be merged with the Master branch using git merge. This will be covered later.

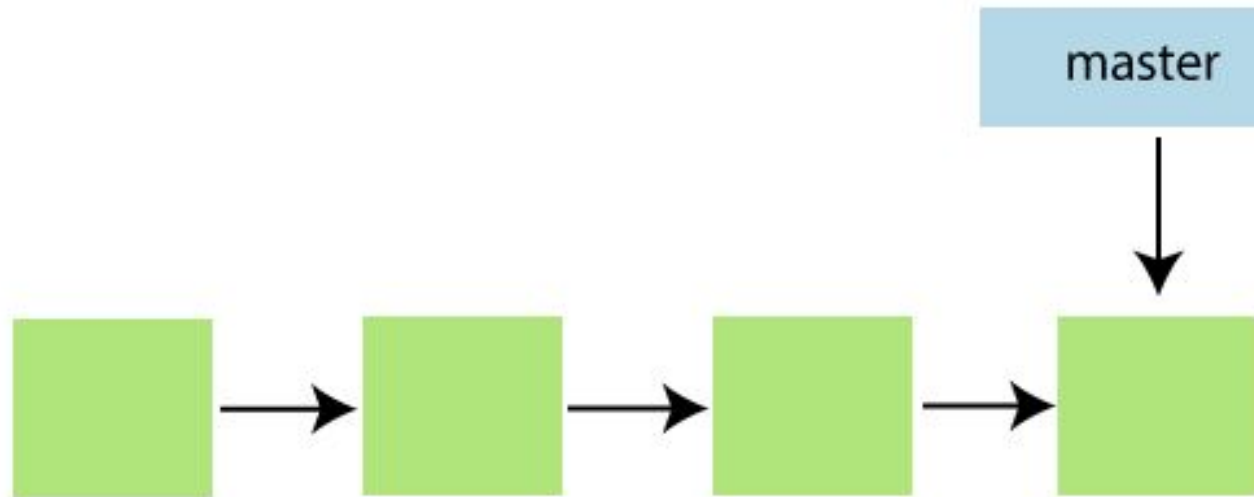


Git Master

Master is a naming convention for Git branch. It's a default branch of Git. After cloning a project from a remote server, the resulting local repository contains only a single local branch. This branch is called a "master" branch. It means that "master" is a repository's "default" branch.

In most cases, the master is referred to as the main branch. Master branch is considered as the final view of the repo. Your local repository has its master branch that always up to date with the master of a remote repository.

becareful with the master. If you edited the master branch of a group project, your changes will affect everyone else and very quickly there will be merge conflicts.

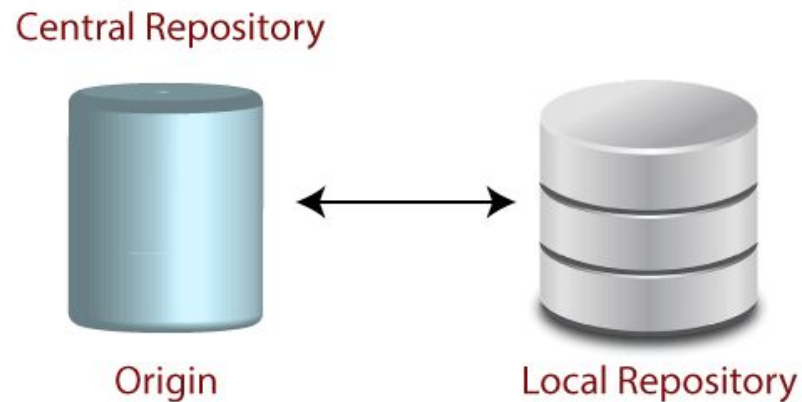


Git Origin

In Git, The term origin is referred to the remote repository where you want to publish your commits. The default remote repository is called **origin**, although you can work with several remotes having a different name at the same time. It is said as an alias of the system.

The origin is a short name for the remote repository that a project was initially being cloned. It is used in place of the original repository URL. Thus, it makes referencing much easier.

Origin is just a standard convention. Although it is significant to leave this convention untouched, you could ideally rename it without losing any functionality.



Branches-operations - 1

Create a New Branch in Local

Create a new branch called **test** using the following command:

```
git branch test
```

This command creates the test branch.

We are still in the context of the master branch. In order to switch to the test branch. use the following command:

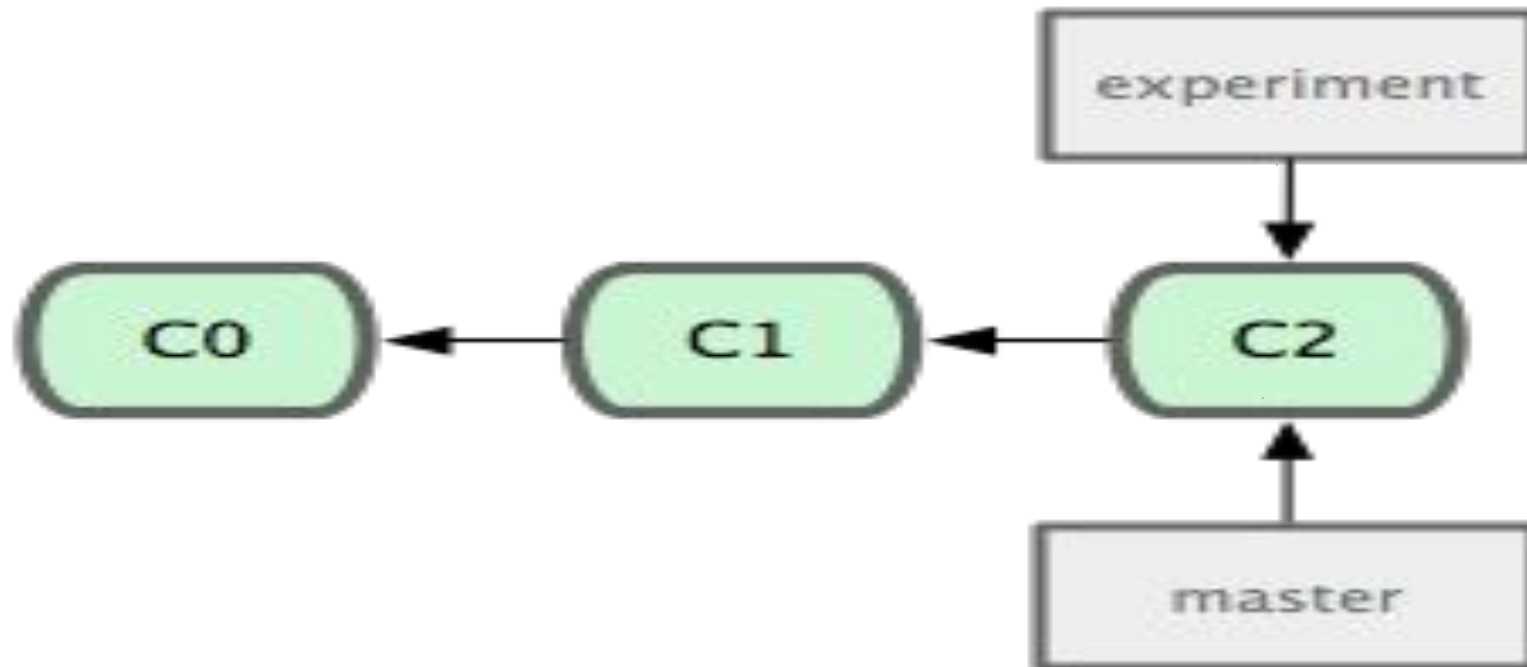
```
git checkout test
```

Now we are in the test branch.

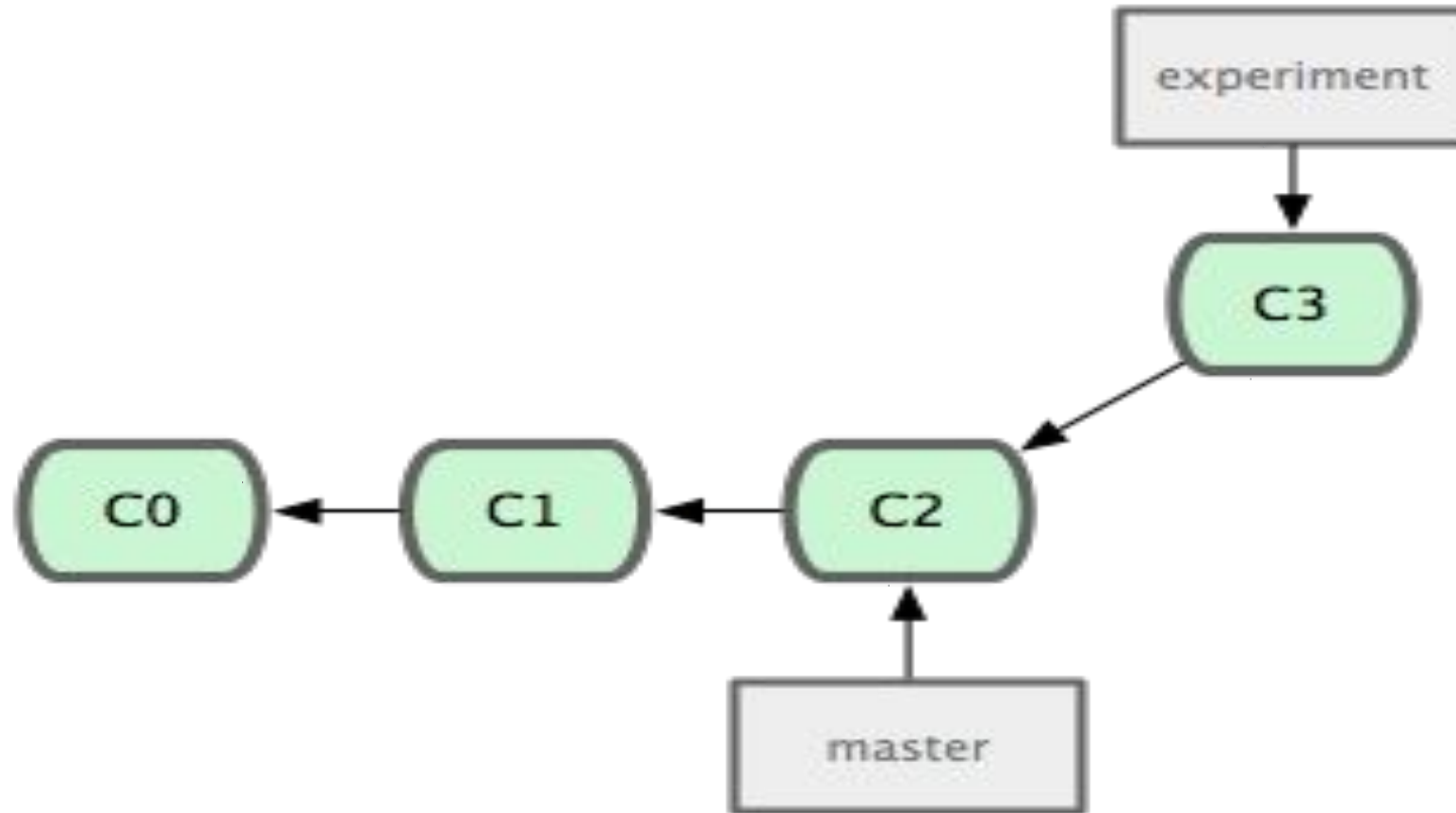
You can list out all the branches in local using the following command:

```
git branch
```

Git creating branches



Commit in Branch



Branches-operations - 2

Do Some Commits in the New Branch

Example: Modify demo.txt by adding the following snippet:

Initial Content Adding more Content Adding some Content from test Branch

Now stage and commit using the following commands:

```
git add demo.txt git commit -m "Test Branch Commit"
```

This commit was done in the Test Branch, and now Test Branch is ahead of Master Branch by 1 commit — as the test branch also includes the 2 commits from the master branch.

You can verify the commit history in Test Branch using:

```
git log
```

Branches-operations - 3 (Merge)

Merging

Currently, Test Branch is ahead of the Master by 1 commit. Let's say that now we want all the code in the Test Branch to be brought back to the Master Branch. This is where git merge is very useful.

In order to merge the code from the test branch into the master branch, follow these steps:

First go back to the master branch:

```
git checkout master
```

Then run the merge command:

```
git merge test
```

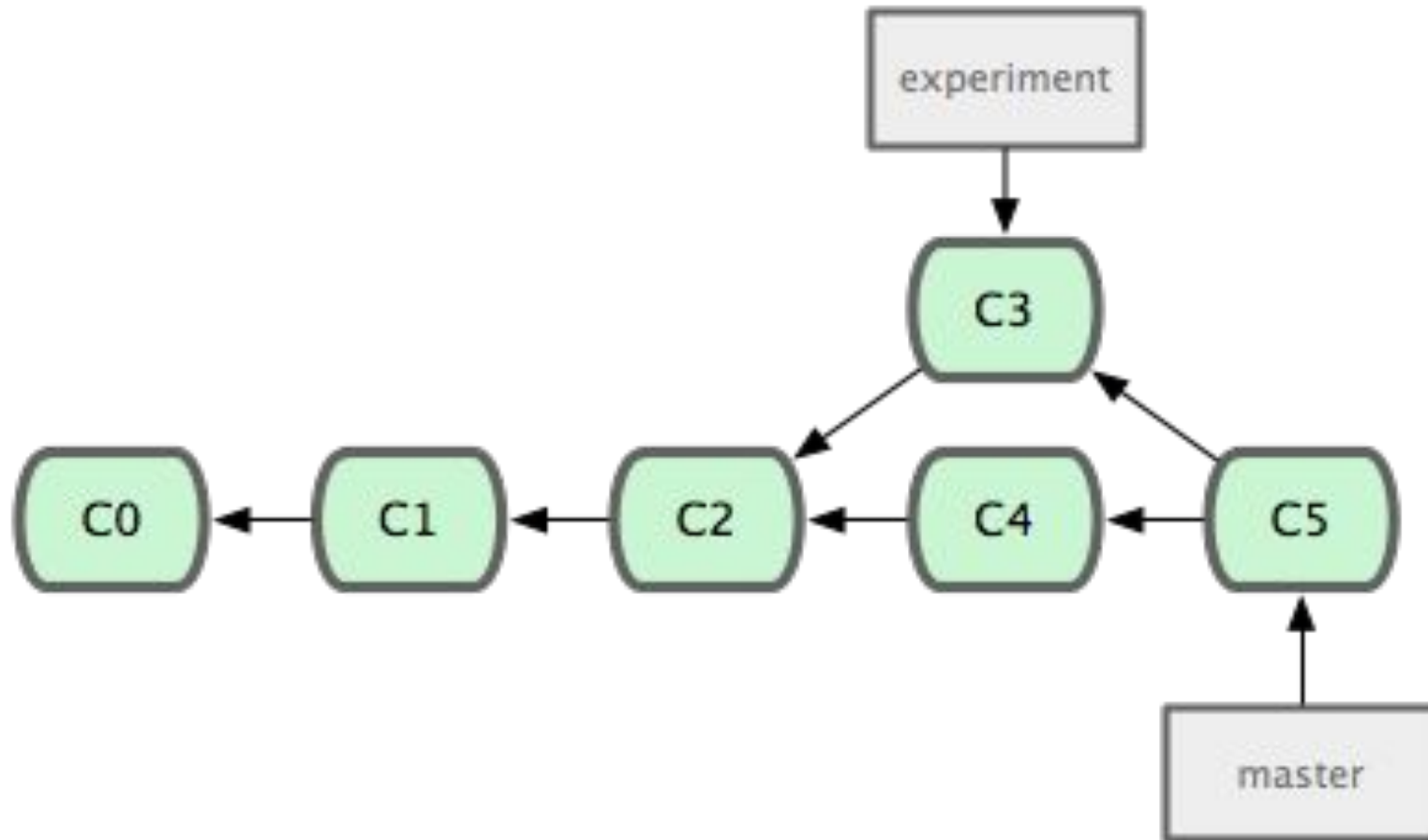
After running these 2 commands, the merge should be successful. In this example, there are no conflicts.

But in real projects, there will be conflicts when a merge is being done. Resolving the conflict is something which comes with experience, so as you work more with Git you will be able to get the hang of resolving conflicts.

Run `git log` now and you will notice that the master also has **3 commits**.

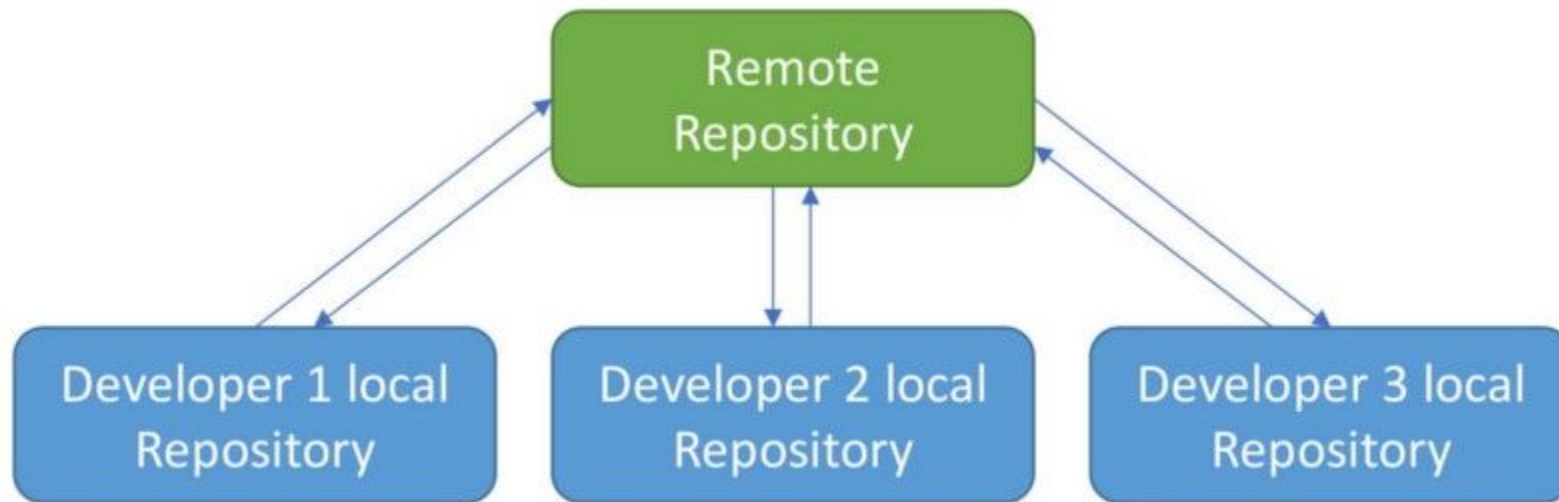
<https://www.freecodecamp.org/news/what-is-git-and-how-to-use-it-c341b049ae61/>

Merging



The Remote Git Repository

Until now, we have been working only in the local repository. Each developer will work in their local repository but eventually, they will push the code into a remote repository. Once the code is in the remote repository, other developers can see and modify that code.



Git vs. GitHub: What's the Difference?

In Simple Terms

Git

is a version control system that lets you manage and keep track of your source code history

Git
Hub

is a cloud-based hosting service that lets you manage Git repositories

Git is not the same as Github

<https://devmountain.com/blog/git-vs-github-whats-the-difference/>

Git and Github comparison [\(reference\)](#)

	Git	GitHub
1.	Git is a software .	GitHub is a service .
2.	Git is a command-line tool	GitHub is a graphical user interface
3.	Git is installed locally on the system	GitHub is hosted on the web
4.	Git is maintained by linux .	GitHub is maintained by Microsoft .
5.	Git is focused on version control and code sharing.	GitHub is focused on centralized source code hosting.
6.	Git is a version control system to manage source code history.	GitHub is a hosting service for Git repositories.
7.	Git was first released in 2005.	GitHub was launched in 2008.
8.	Git has no user management feature .	GitHub has a built-in user management feature .
9.	Git is open-source licensed.	GitHub includes a free-tier and pay-for-use tier.
10.	Git has minimal external tool configuration.	GitHub has an active marketplace for tool integration.
11.	Git provides a Desktop interface named Git Gui.	GitHub provides a Desktop interface named GitHub Desktop.
12.	Git competes with CVS, Azure DevOps Server, Subversion, Mercurial, etc.	GitHub competes with GitLab, Git Bucket, AWS Code Commit, etc.

Common Git commands-1

init

The “git init” command creates a new Git repository. It can be used to convert an existing, unversioned project to a Git repository or initialize a new empty repository.

clone

The “git clone” command copies an existing Git repository, e.g. from remote location github.com. Cloning automatically creates a remote connection called origin pointing back to the original repository.

add

The “git add” command adds a change in the working directory to the staging area. It tells Git that you want to include updates to a particular file in the next commit.

commit

The “git commit” command commits the staged snapshot to the project history. Snapshots are committed to the local repository, and this requires absolutely no interaction with other Git repositories.

<https://blogs.sap.com/2019/07/18/basic-operations-of-git/>

Common Git commands-2

push

The “git push” command is to **transfer** commits from **your local repository** to a **remote repo**.

branch

A branch represents an **independent** line of development. New **commits** are recorded in the history for the current branch, which results in a fork in the history of the project. The “git branch” command lets you create, list, rename, and delete branches.

fetch

The “git fetch” command imports commits from a **remote repository** into **your local repo**. The resulting commits are stored as remote branches instead of the normal local branches that we’ve been working with. This gives you a chance to review changes before integrating them into your copy of the project.

checkout

Checking out a branch **updates** the files in the **working directory** to match the version stored in that branch, and it tells Git to record all new commits on that branch.

Common Git commands-3

pull

The “git pull” command is to merge upstream changes into your local repository. you can also do this with “git fetch” followed by “git merge”.

merge

The “git merge” command is to merge all committed changes on a specified branch into current branch.

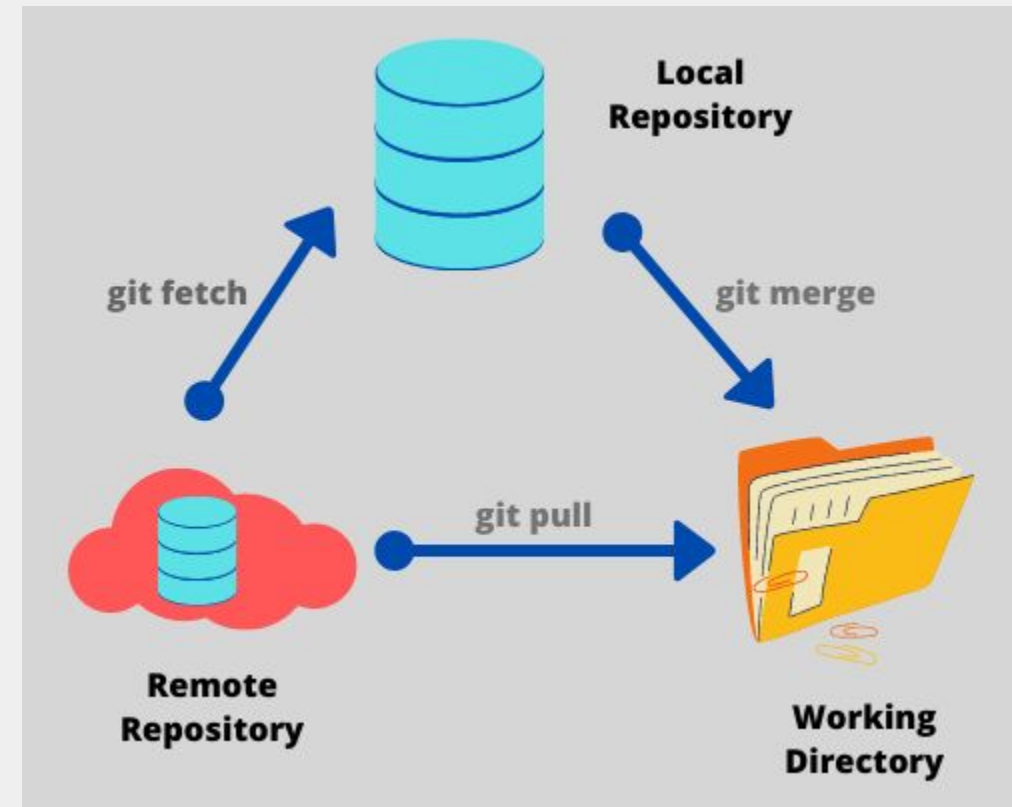
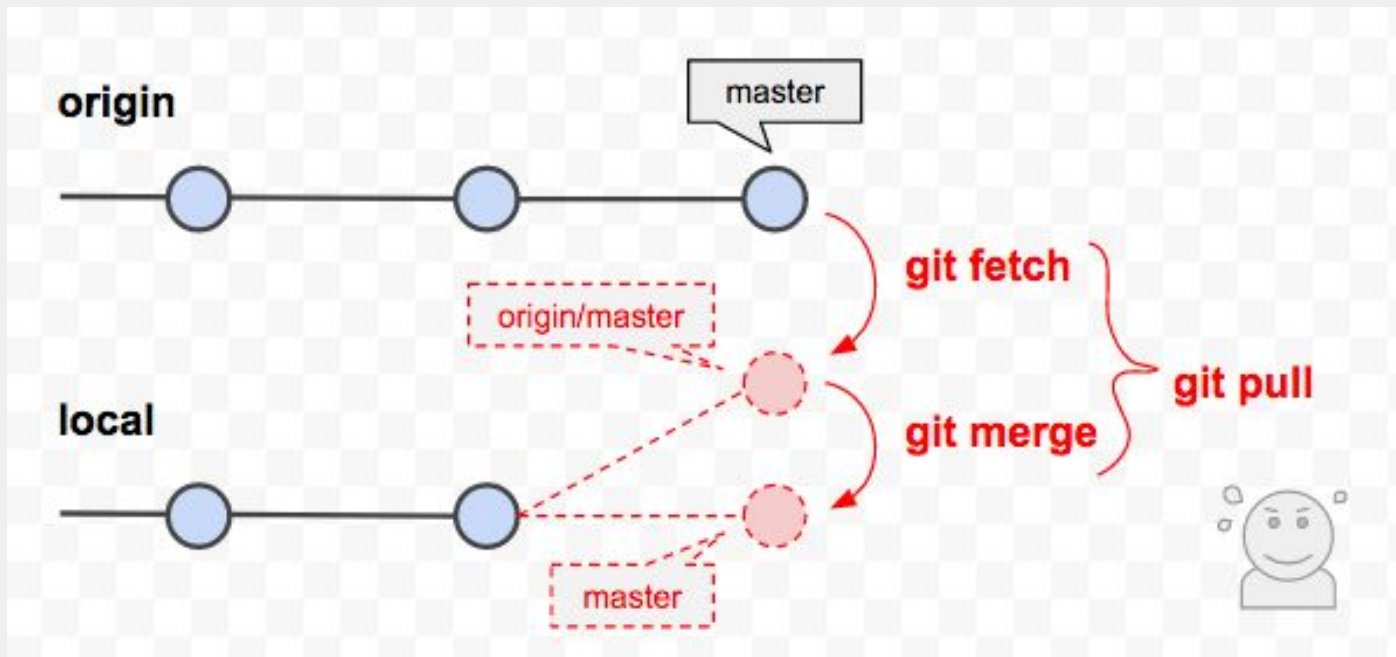
rebase

Rebasing is the process of **moving a branch** to a new base commit. Internally, Git accomplishes this by creating new commits and applying them to the specified base, it's literally rewriting your project history. The commits that were previously saved into the temporary area are then reapplied to the current branch, one by one, in order.

..... continue in next slide>>

Difference between (Pull and Merge)

- pull = fetch + merge



Difference between (Pull and Merge)-2

GIT PULL VERSUS GIT FETCH

GIT PULL

A GIT command that downloads latest changes from the remote repository and automatically merge those changes in the local repository

Downloads the changes in the remote repository and merges those changes and store them on the local repository

Git Pull is similar to GIT Fetch followed by GIT Merge

Allows the developer to bring changes to the local code repository and to update it

GIT FETCH

A GIT command that pulls down the code from the remote repository to tracking branches in the local repository

Retrieves the changes made in the remote repository without merging them

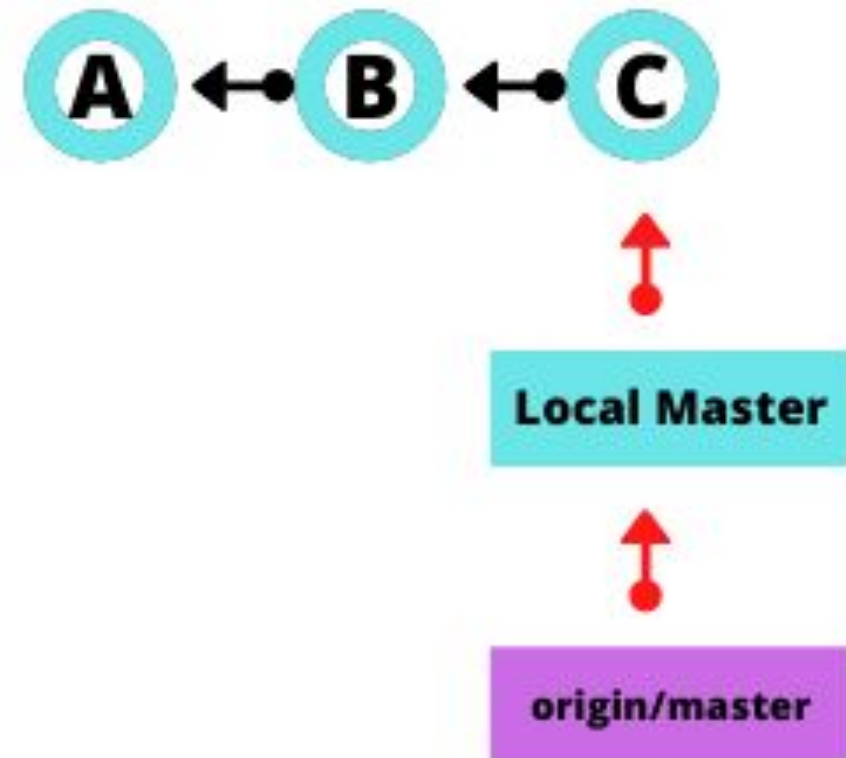
GIT Fetch is a simple command

Allows the developer to know the commits pushed by other developers before integrating them with the local repository

https://medium.com/@sabbirhossain_70520/git-fetch-vs-git-pull-691823ed4239

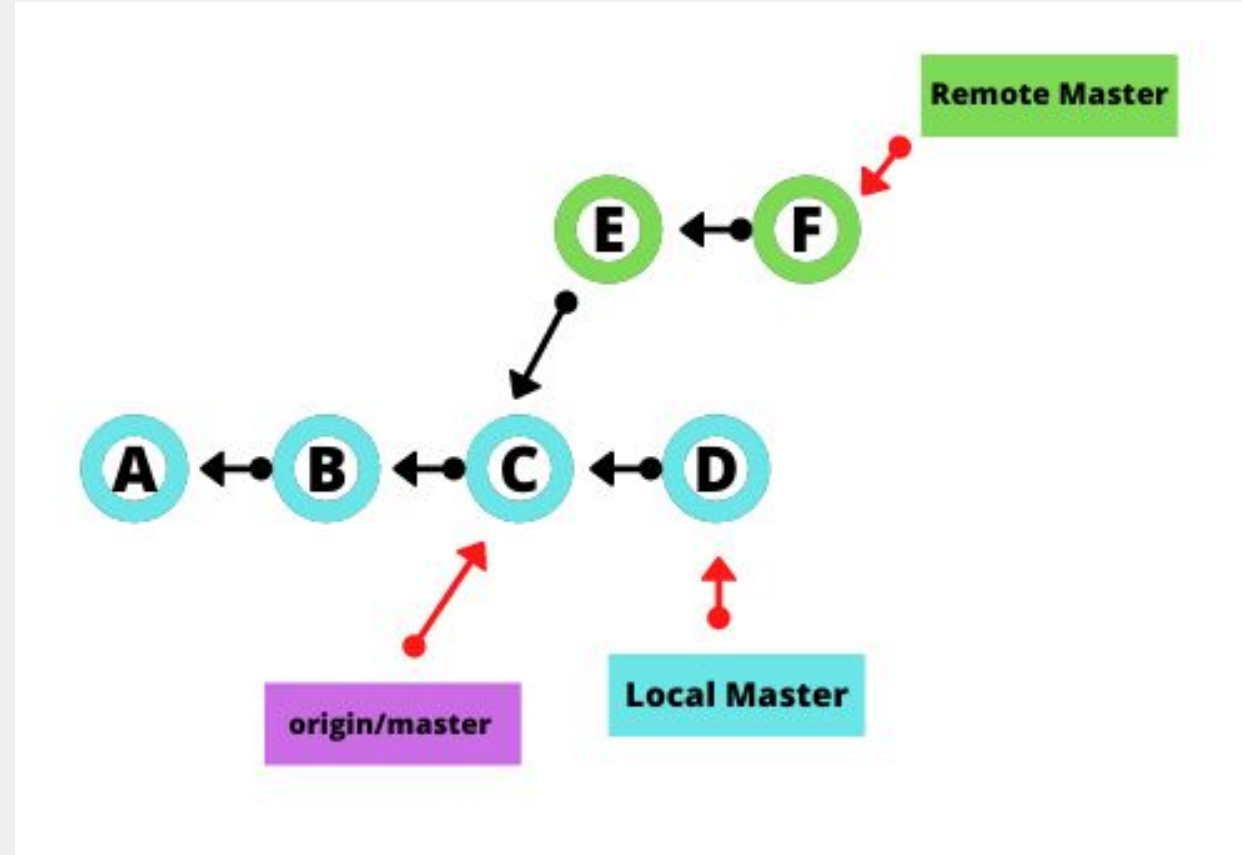
Example of remote tracking-1

- 1-Suppose we cloned a remote repository having three commits. We will have the remote-tracking branch reference origin/master and the local master branch reference pointing at commit point C



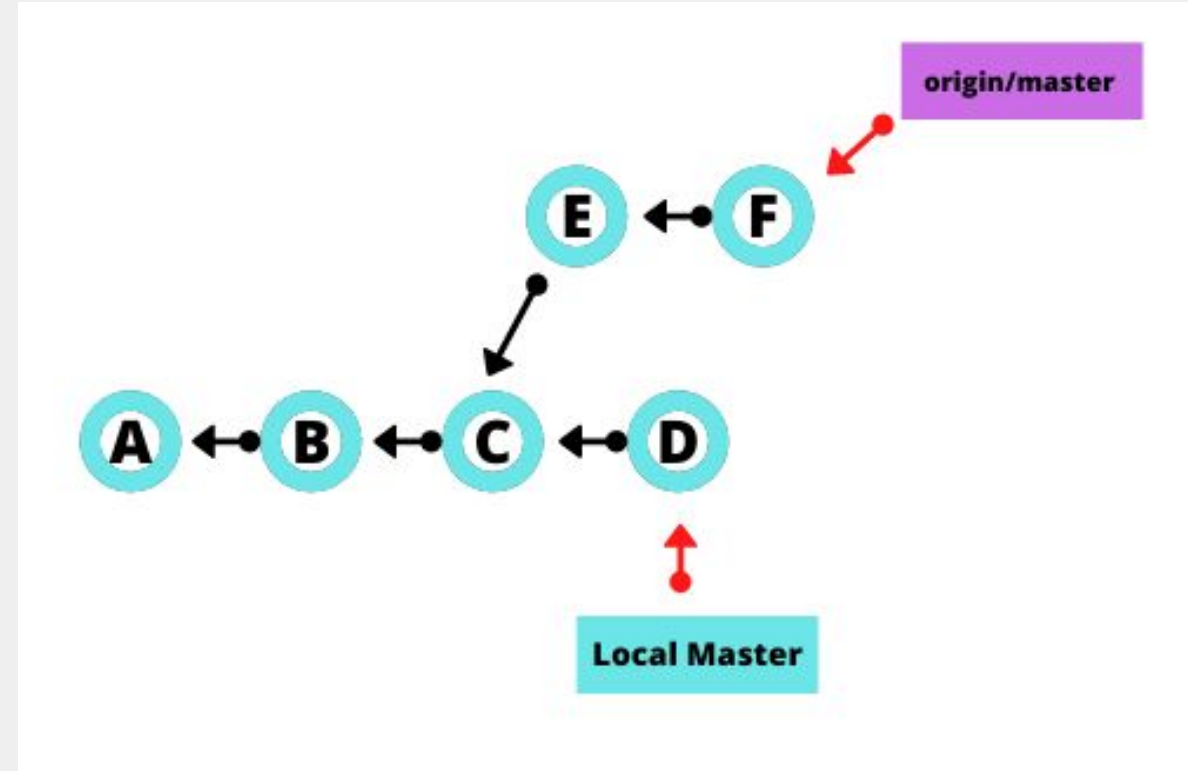
Example of remote tracking-2

- 2-Next, we add one commit to our local master branch and some other developer adds two more commits to the remote repository



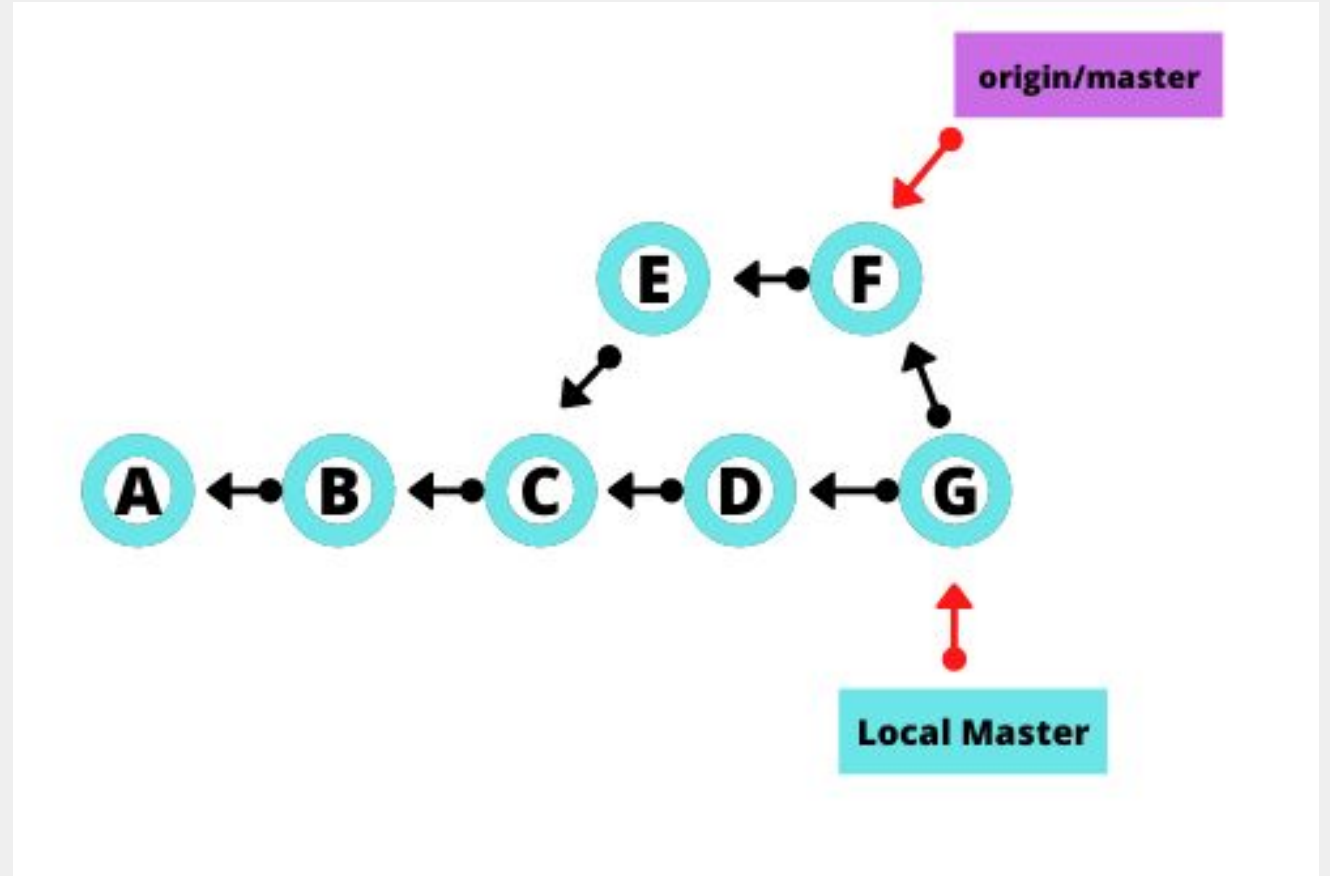
Example of remote tracking-3

- 3-Now if we run the **Git Fetch** command the two new commits of the remote repository will be added to the remote-tracking branch.



Example of remote tracking-4

- 4-But instead, if we run the Git Pull command the two commits will be fetched just like in the above step but will also be added to the local master branch in the form of a new merge commit point G. Thus our local working branch will be altered.



rebase (example)

Assume the following history exists and the current branch is "topic":

```
    A---B---C topic
   /
  D---E---F---G master
```

Now executes the following command.

```
git rebase master
```

The branches will be:

```
        A'---B'---C' topic
       /
      D---E---F---G master
```

It is possible that a merge failure will prevent this process from being completely automatic. You will have to resolve any such merge failure and run `git rebase --continue`. Another option is to bypass the commit that caused the merge failure with `git rebase --skip`. To check out the original <branch> and remove the `.git/rebase-apply` working files, use the command `git rebase --abort` instead.

Getting started

- Set your identity
 - `$ git config --global user.name "John Doe"`
 - `$ git config --global user.email jdoe@example.com`
- Get help
 - `$ git help <verb>`

Creating a new repository

- `$ git init`
- Creates a new (empty) repository in the current directory

Copying a repository

- For this class, your instructor will create a repository for you,
you will just need to copy it from GitHub to your computer
using the following command:
 - `$ git clone <repository>`
 - Creates a copy of `<repository>` in the current directory

Staging files

- As you work, you will create new files and modify existing files, when you are satisfied with your changes, you can stage them for commit with:
 - `$ git add <file_pattern>`

Committing changes

- *Commits* create a new version in the repository
- Include a commit message describing the new version
- `$ git commit -m <msg>`

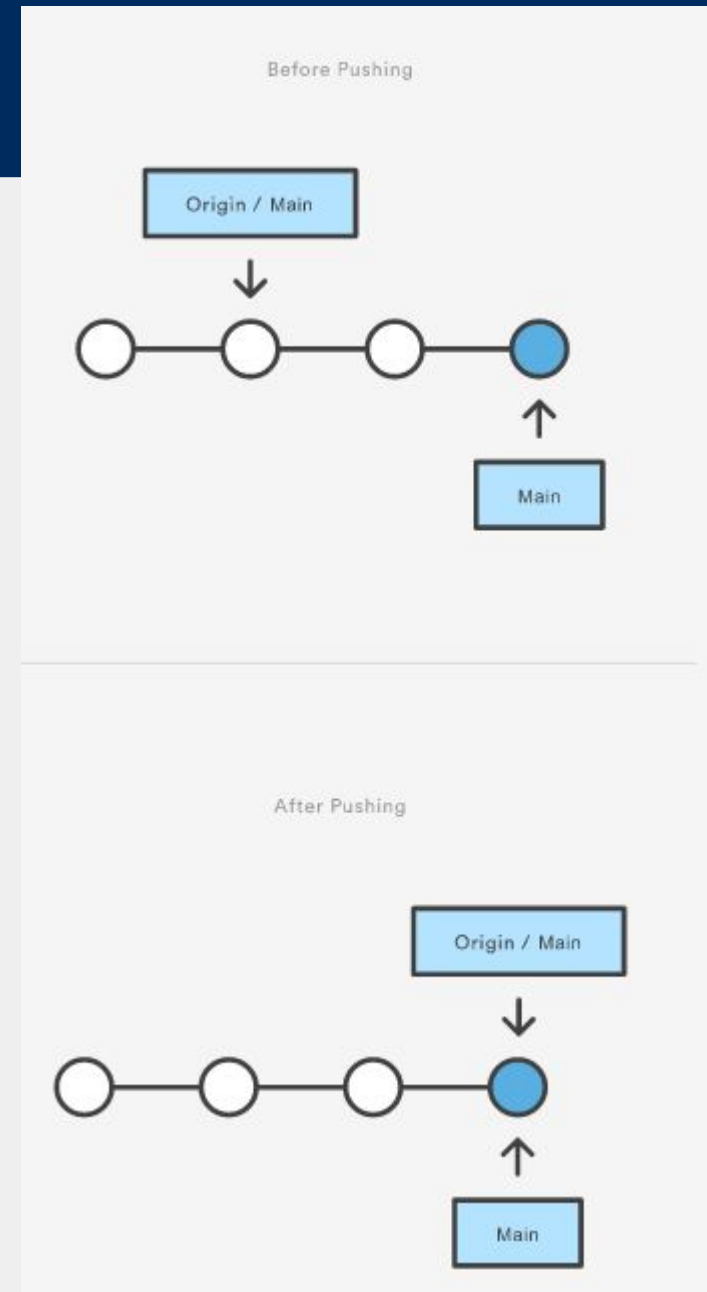
Checking working directory status

- `$ git status`
- Reports:
 - Files in the working directory that are not tracked
 - File modifications not yet staged for commit
 - File additions and modifications staged for commit

Pushing changes

- Push the specified branch to **remote**, along with all of the necessary commits and internal objects. This creates a local branch in the destination repository.
- `$ git push <remote> <branch>`

<https://www.atlassian.com/git/tutorials/syncing/git-push>



Steps for common tasks-1

Commit changes to remote master directly

- Make some changes to file01.txt in your working directory.
- `git add file01.txt`
- `git commit -m "Commit M1"`
- `git push -u origin master`

Steps for common tasks-2

Create a branch and commit changes to it

- Open a new branch named feature01: `git checkout -b feature01`
- Make some changes to file01.txt
- `git add file01.txt`
- `git commit -m "Commit feature01"`
- `git push -u origin feature01`

Merge a branch into master branch

- Suppose you have committed all changes to branch feature01
- `git checkout master`
- `git merge feature01`
- `git push -u origin master`

Steps for common tasks-3

Delete a branch

- Delete a local branch: `git branch -d feature01`
- Delete a remote branch: `git push origin --delete feature01`

Rebase a branch on last commit on master

- `git checkout feature01`
- `git rebase master`

Merge a branch by rebasing it on master

- `git checkout feature01`
- `git rebase master`
- `git checkout master`
- `git merge feature01`
- `git push -u origin master`

Get latest changes on a remote branch

- `git checkout feature01`
- `git pull origin`

Overviewing commit history

- \$ git log
- Lists commits made to the current repository

(.gitignore) file

.gitignore files are present in the root directory of your repository.

You can specify files to ignore when staging, so your `git status` commands don't blow up the command line if you have a lot of binaries or photos that you will never intend on committing.

Resource:

<https://help.github.com/articles/ignoring-files/>

Specifying License in Github project

<https://www.digitalocean.com/community/tutorials/understanding-open-source-software-licenses>

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

☐ Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

☒ Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▼

This is the default name in your [settings](#).

License

Filter licenses...

- ✓ None
- Apache License 2.0
- GNU General Public License v3.0
- MIT License
- BSD 2-Clause "Simplified" License
- BSD 3-Clause "New" or "Revised" License
- Boost Software License 1.0
- Creative Commons Zero v1.0 Universal
- Eclipse Public License 2.0
- GNU Affero General Public License v3.0
- GNU General Public License v2.0
- GNU Lesser General Public License v2.1

Term

 [Contact GitHub](#)

Creating your first repo (repository)



Go to github.com, and click on “New repository”

The repository name (not your username) will be in the form `cs<ClassNumber><LastName>`. No underscores, hyphens or spaces please; there are a lot of students, and this helps grading.

- `cs480Lopez` or `cs680Lopez`

Download your repository to a common directory you will be using. This will be your workspace from now on.

```
tennisgazelle@GazelleLaptop:~/workspace$ git clone https://github.com/TennisGazelle/cs480Lopez.git
Cloning into 'cs480Lopez'...
Username for 'https://github.com': TennisGazelle
Password for 'https://TennisGazelle@github.com':
remote: Counting objects: 247, done.
remote: Total 247 (delta 0), reused 0 (delta 0), pack-reused 247
Receiving objects: 100% (247/247), 365.49 KiB | 0 bytes/s, done.
Resolving deltas: 100% (114/114), done.
Checking connectivity... done.
tennisgazelle@GazelleLaptop:~/workspace$ ls
cs480Lopez
```

Pro Tip! – Branching and Pull Requests.

If you are comfortable enough with github, you might want to consider branching during group projects to separate work tasks amongst people.

Branches are *deviations* in the commit tree which get requested to be joined to the master branch through merging, or formally, through pull requests.

These are good for feature management, but *merge conflicts* are something that may occur. **These are common both in industry, so learn to deal with them now!**

Resource:

<https://www.atlassian.com/git/tutorials/comparing-workflows>

What is Markdown?

- Markdown is a way to style text on the web. You control the display of the document; formatting words as bold or italic, adding images, and creating lists are just a few of the things we can do with Markdown. Mostly, Markdown is just regular text with a few non-alphabetic characters thrown in, like # or *.
- <https://guides.github.com/features/mastering-markdown/>
- Example:

```
It's very easy to make some words **bold** and other words  
*italic* with Markdown. You can even  
[link to Google!](http://google.com)
```

It's very easy to make some words **bold** and other words *italic* with
Markdown. You can even [link to Google!](http://google.com)

GitHub Flavored Markdown

Markdown is a lightweight markup language with simple syntax for creating formatted text. Put more simply, Markdown makes it easy to create formatted text for a variety of uses not including GitHub, such as blogging, certain messaging platforms, documentation, and specific to what you're interested in today, README files.

GitHub uses its **own version of Markdown**, referred to as **GitHub Flavored Markdown**. It also supports rendering Markdown (.md) files

<https://learn.adafruit.com/excellent-github-profile/markdown-basics>

Example : Markdown- Headings

Markdown	HTML	Rendered Output
# Heading level 1	Heading level 1	Heading level 1
## Heading level 2	Heading level 2	Heading level 2
### Heading level 3	Heading level 3	Heading level 3
#### Heading level 4	Heading level 4	Heading level 4
##### Heading level 5	Heading level 5	Heading level 5
##### Heading level 6	Heading level 6	Heading level 6

Check
Attached file (MarkdownSheet.pdf)

More references:

<https://www.markdownguide.org/basic-syntax/>

<https://learn.adafruit.com/excellent-github-profile/markdown-basics>

<https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Practice- Contributing to Project

- 1- Use the **first 6 steps** in the following article:
<https://codeburst.io/a-step-by-step-guide-to-making-your-first-github-contribution-5302260a2940>
to clone the following repository:
<https://github.com/waleedcs2000/javascript-for-kids>
(a word guessing game like “hangman” to guess secret words)
- 2- Let the new branch name with you Name.
- 3- add file named “student.md” with your Name in ENGLISH
- 4- edit the repository to use HTML tags GUI instead of Prompt and Alert methods in the existing code.
- **5-Send pull request** to project hoster (waleedcs2000)

References

- <https://www.toptal.com/open-source/developers-guide-to-open-source-licenses>
- Based on online presentations By :
 - BIRBAL JAT, PGT-Comp.Sc.
 - yasmin maki